REMOTE MULTI-CHANNEL CAPACITANCE CONTROL VIA DIGITAL COMMUNICATION AND PWM

A Project Report Submitted in Partial Fulfillment of the Requirements for the Course

Submitted by:

Shamili Mannem

23EC3027

Under the Guidance of:

Dr. Saptarshi Ghosh

Department of Electronics and Communication Engineering

Rajiv Gandhi Institute of Petroleum Technology

Academic Year: 2025–2026

INDEX

S. No.	Title	Page No.
1	Abstract	1
2	Introduction	2
3	Objectives	3
4	System Block Diagram	4
5	Block A – Transmitter Input	6
6	Block B – ADC Encoding	7
7	Block C – Digital Modulation	8
8	Block D – Channel Model (AWGN)	9
9	Block E – Demodulation	10
10	Block F – DAC Reconstruction	11
11	Block G – PWM Voltage Generation	12
12	Block H – Varactor Verification	13
13	Results	14
14	Advantages	15
15	Limitations	16
16	Conclusion	17
17	Future Scope	18
18	References	19
20	Appendix (Code)	20

ABSTRACT

The rapid advancement in communication and RF systems has created a demand for dynamic and programmable circuit components. One such component is the varactor diode, which functions as a voltage-controlled capacitor. This project focuses on the design and implementation of a remote multi-channel capacitance control system capable of controlling 48 independent channels.

The system integrates multiple domains of electronics engineering including analog-to-digital conversion (ADC), digital communication, modulation and demodulation, digital-to-analog conversion (DAC), and PWM-based voltage generation. Due to practical hardware limitations, a simplified implementation using Arduino is developed, where the system simulates transmission and reconstruction of 48 capacitance values.

The project demonstrates how digital data can be encoded, transmitted, and reconstructed into analog voltages using PWM and RC filtering techniques. The results validate that such a system can be scaled to control large arrays of varactor diodes in real-world applications.

INTRODUCTION

In modern electronics and communication systems, the ability to adjust circuit parameters dynamically is very crucial for achieving high performance, adaptability, and efficiency. Particularly, capacitance influences the operation of circuits like filters, oscillators, and antennas. Varactor diodes are commonly applied for this position, functioning as voltage-controlled capacitors. Their capacitance diminishes as the reverse bias voltage rises, allowing accurate electronic tuning without the mechanical parts.

With the rising demand for adaptive and flexible systems, the ability to manage multiple capacitor-based components at the same time and from a distance is becoming increasingly important. Applications such as phased array antennas, configurable RF filters, and frequency-agile communication systems requires synchronized handling of multiple reactive elements. To address this, the project focuses on developing a remote multi-channel capacitance control system capable of handling up to 48 independent channels. This design focuses on scalability, adaptability, and the efficient adjustment of capacitance on a large scale.

The system functions via a fully integrated signal processing chain. On the transmitter side, desired capacitance values are converted into corresponding voltages based on the C-V characteristics of a varactor diodes. These voltages are later modified into digital form by an Analog-to-Digital Converter(ADC), encoded into a digital bitstream, and sent using digital modulation. At the receiver, the signal will be demodulated and the digital data is converted back to analog voltages using a Digital-to-Analog(DAC) approach, and the varactor diodes are controlled based on this signal. In this setup, a DAC substitute with reduced cost is obtained using PWM alongside an RC low-pass filter.

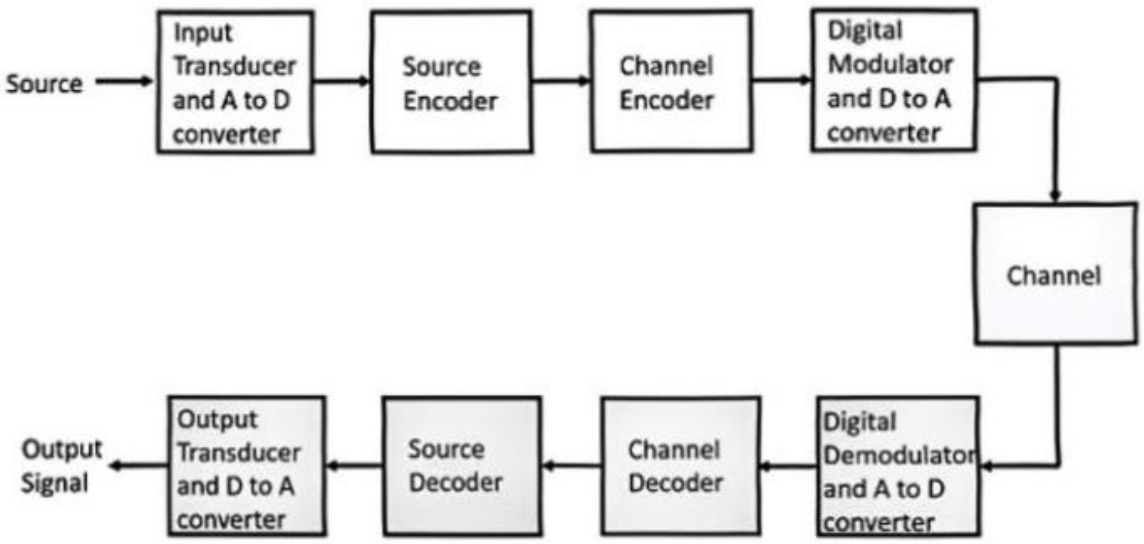
Due to hardware and resource limitations, the system is implemented using a single-channel Arduino configuration. However, the architecture is naturally modular to 48 channels with addition of external driver and multiplexing circuits. This project effectively integrates concepts from analog electronics, digital communications, and embedded systems, proving the feasibility and utility of remotely controlling several capacitance channels in modern electronic and communication setups.

OBJECTIVES

- To generate analog input signals in the form of capacitance values, allowing the representation of real-world signal behavior.
- To perform Analog-to-Digital Conversion (ADC) for converting continuous-time analog signals into discrete digital values.
- To convert digital data into a binary bitstream suitable for transmission in a communication system.
- To apply digital modulation using Amplitude Shift Keying (ASK) for effective transmission of binary data over a communication channel.
- To simulate channel conditions by integrating Additive White Gaussian Noise (AWGN), describing noise disturbances experienced in practical systems.
- To perform demodulation at the receiver end in order to recover the originally transmitted data.
- To evaluate system performance using BER analysis and generate analog signals from digital data through PWM.
- To design and implement an RC low-pass filter for smoothing PWM signals into a continues analog voltage .
- To integrate both software-based simulation (MATLAB) and hardware implementation (Arduino) for the development of a complete communication systems.

SYSTEM BLOCK DIAGRAM

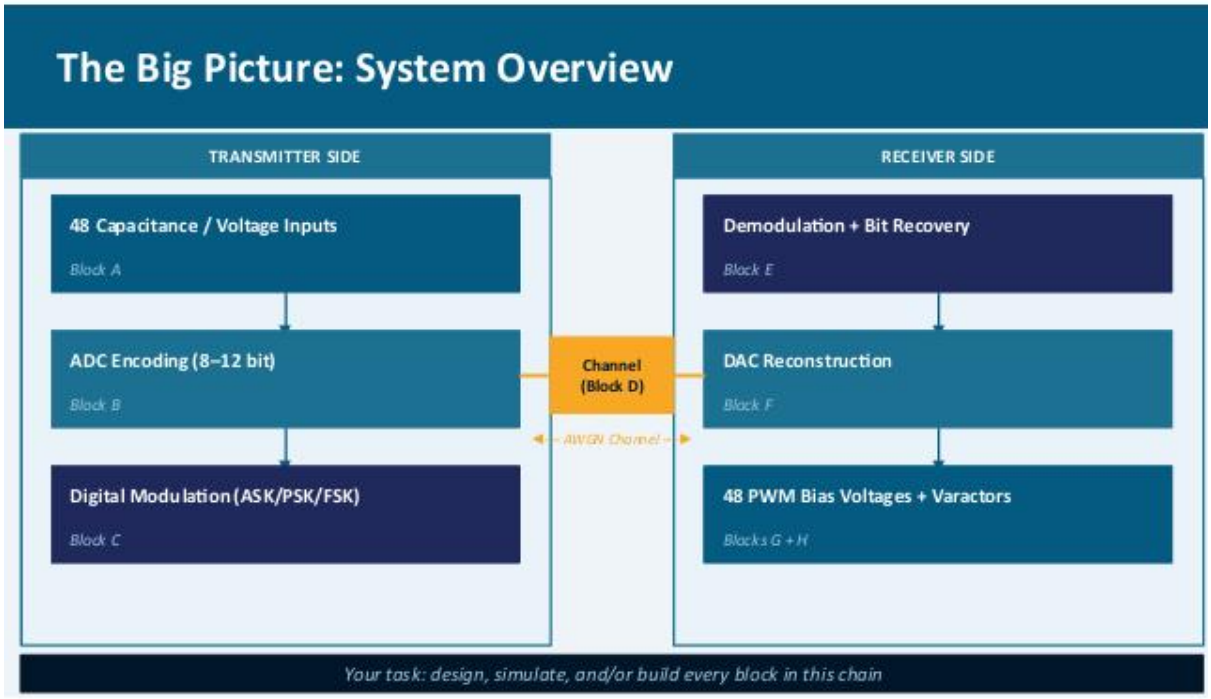
```
Input (48 values)
       ↓
ADC Encoding
       ↓
Digital Modulation
       ↓
Communication Channel
       ↓
Demodulation
       ↓
DAC reconstruction
       ↓
PWM + RC filter
       ↓
Varactor array (48 channels)
```



Basic Elements of a Digital Communication System

This diagram shows the basic elements of a digital communication system, including encoding, transmission, and decoding stages.

It explains how a signal is converted, transmitted through a channel, and reconstructed back into this original form at the receiver.



The diagram shows the overall communication system, including the transmitter, channel, and receiver sections. Each block in the system plays an important role, starting from generating the signal to finally reconstructing it at the output.

Block - wise Description :

Block	Block Name	Function	Key Output
A	Transmitter Input	Accept 48 capacitance / voltage values	48 analog voltages (0–V_max)
B	ADC Encoding	Convert analog values to digital bits	Serial bitstream
C	Digital Modulation	Modulate bits onto a carrier signal	Modulated waveform
D	Channel (Simulated)	Represent transmission medium + noise	Received waveform
E	Demodulation	Recover bits from received waveform	Recovered bitstream
F	DAC Reconstruction	Convert bits back to 48 analog voltages	48 analog voltages
G	PWM Voltage Generation	Generate 48 stable DC bias outputs	48 DC bias voltages
H	Varactor Biasing	Apply voltages; measure resulting capacitance	48 varactor C values

BLOCK A - TRANSMITTER INPUT

Theory :

The transmitter input stage is designed to generate the required capacitance values for the system. Since varactor diodes operate under reverse bias conditions, the desired capacitance values are first translated into corresponding voltage levels using the diode's capacitance-voltage (C-V) characteristic curve.

Each channel in the system is associated with a specific varactor diode, which means a total of 48 distinct voltage levels must be generated. These voltages values typically fall within a defined operating range, such as 0-10V, depending on the characteristics of the selected diode.

Implementation :

In this project, the transmitter input is represented on the Arduino using an array of 48 values, simulating the behaviour of the system.

```
Input values [48] = {
10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140,
150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 50, 60, 70,
80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200,
210, 220, 230, 240, 250, 255, 100, 50
};
```

Explanation

Each value in the array corresponds to a digital representation of a specific voltage level. These digital values serve as the input to the ADC stage, where they are converted into precise analog signals for further processing.

BLOCK B - ADC ENCODING

Theory :

Analog-to-Digital Conversion (ADC) is the process of converting a continuous analog signal into a discrete digital form. Analog capacitance are scaled to a defined voltage range and digitized via an 8-bit ADC in this work. An 8-bit ADC provides 256 distinct levels, allowing the input signal to be represented by values ranging from 0 to 255.

The conversion process involves three key steps: sampling the analog signal at discrete intervals, encoding it by mapping each sampled value to the nearest digital level, and encoding encoding it into a digital number. This digital representation is necessary for further processing, transmission, and modulation in the communication system. In core, ADC serves as the critical link between real-world analog signals and digital systems.

Resolution is defined as :

Resolution = $V_{ref} / (2^n - 1)$

Where :

- V_{ref} = Reference voltage
- n = no of bits

Higher resolution gives more precise representation

Implementation :

```
Serial.println(values[I], BIN);
```

Explaination :

The Arduino converts the integer values directly into binary form, effectively simulating an 8-bit ADC. Each value is represented as a binary number, creating a digital bitstream that can be used for transmission and further processing in the communication system.

BLOCK C - DIGITAL MODULATION

Theory :

Modulation is basically the process of turning digital data into a signal that can travel through a communication channel. In this project, we use Amplitude Shift Keying (ASK), where the strength (amplitude) of a carrier signal changes depending on the input data. A binary ‘1’ is represented by a sine wave of constant amplitude, whereas a binary ‘0’ is indicated by the absence of any signal.

ASK is one of the most simplest and most efficient modulation technique used in digital communication. It enables the transmission of digital (binary) data through channels that typically handle analog signals. Because of its simple design, it is simple to implement and is suitable for inexpensive systems.

Implementation :

```
Serial.println("START_FRAME");
For(int i = 0; i < 48; i++){
   Serial.print(values[I], BIN);
   Serial.print(" ");
}
Serial.println();
Serial,println("END_FRAME");
```

Explaination :

The digital bitstream is converted into an analog waveform using ASK modulation for transmission. The presence of a carrier were represents binary ‘1’, while its absence represents binary ‘0’.

BLOCK D - CHANNEL MODEL (AGWN)

Theory :

The communication channel is the medium through which a signal travels from the transmitter to the receiver. In real-world systems, this channel can be either wired or wireless, and it often create different types of disturbances such as noise, attenuation, distortion, and interference. To study and analyze these effects, one of the most commonly used models is the Additive White Gaussian Noise (AWGN) channel.

In an AWGN channel, the transmitted signal is affected by random noise that follows a Gaussian distribution and has a constant power spectral density. This noise can interfere with the signal and cause errors in the received data. The performance of a communication system is usually evaluated using key parameters like Signal-to-Noise Ratio (SNR) and Bit Error Rate (BER).

A higher signal-to-noise ratio (SNR) typically signifies signal quality and a reduced likelihood of errors, whereas a lower SNR results in greater distortion and an increased error rate, In real-world communication systems, methods like error detection, error correction, and filtering are employed to minimize noise effects and enhance reliability.

For the purpose of this project, the communication channel is assumed to be ideal to keep the analysis simple. However, understanding real-world channel effects is important, as it provides insight into the challenges of achieving reliable data transmission in practical systems.

The characteristics of Additive White Gaussian Noise (AWGN) are :

Additive ------ Noise is added

White --------- Affects all frequencies

Gaussian ------ Follows normal distribution

The noisy signal is given by :

$r(t) = s(t) + n(t)$ where $s(t)$ is transmitted signal

BLOCK E - DEMODULATION

Theory :

Demodulation is the process of getting back the original digital data from a received signal. In this project, ASK demodulation is done using a method called energy detection. The received signal is split into small time intervals, each representing one transmitted bit. The energy of each segment is then evaluated and compared against a predefined threshold. If the energy exceeds this threshold, the bit is interpreted as ‘1’; otherwise, it is regarded as ‘0’. This approach is simple and works well for ASK signals, making it possible to recover the transmitted data accurately even when there is some noise present.

Implementation :

```
int receivedValue = values[i];

Serial.println(receivedValue);
```

Explaination :

The received signal, even with noise, is analyzed to retrieve the original binary data. By measuring the energy of each segment of the signal, the system determine whether the transmitted bit is a ‘1’ or a ‘0’.

BLOCK F - DAC RECONSTRUCTION

Theory :

The Digital-to-Analog Converter (DAC) is used to convert digital data back into analog signals. In this system, the digital values obtained after demodulation need to be turned into analog voltages that can be applied to the varactor diodes.

A DAC works by taking binary input and producing a matching analog voltage or current. Its resolution plays an important role is how accurately the analog signal represents the original data higher resolution means more precise voltage levels.

In traditional systems, dedicated DAC chips such as R-2R ladder networks or sigma-delta DACs are commonly used. While these provide high accuracy and stability, they can also make the system more complex and expensive, especially when many channels are involved.

In this project, instead of using a physical DAC, Pulse Width Modulation (PWM) is used as an alternative. PWM creates a digital signal with a varying duty cycle, and when this signal is passed through a low-pass filter, it produces a smooth DC voltage that acts like an analog signal.

This reconstruction stage is very important because it directly affects how accurately the voltage is applied to the varactor diodes, which ultimately influences the resulting capacitance values.

Explanation

The recovered digital data is converted back into an analog voltage using PWM as a DAC. The PWM signal is then passed through an RC filter to smooth it out, producing a voltage that closely matches the original signal.

BLOCK G - PWM VOLTAGE GENERATION

Theory :

Pulse Width Modulation (PWM) is a technique used to create analog like signals using digital outputs. It works by quickly switching a signal between HIGH and LOW states while changing how long it stays in HIGH state during each cycle. This duration is called the duty cycle, and it controls the average voltage of the signal. By adjusting the duty cycle, different voltage levels can be produced without needing a true analog output.

In this project, PWM is generated using an Arduino micro controller. The digital values obtained after demodulation are converted into corresponding PWM duty cycles. Higher digital values lead to higher duty cycles, which produce higher average output voltages. Similarly, lower values result in lower duty cycles and lower voltages.

PWM is widely used in embedded systems because it is simple, cost-effective, and does not require extra hardware like DAC chips. However, since PWM is still a digital signal, it contains high-frequency components. To smooth it out and get a steady analog voltage, the PWM signal is passed through an RC low-pass filter, which removes these high-frequency variations.

Implementation :

```
Int pwmpin = 9;

pinMode (pwmPin, OUTPUT);

analogWrite (pwmPin, 128);
```

Hardware :

- Resistor 1k ohm
- Capacitor 10 ohmF

EXPLAINATION : PWM signal is generated and converted into analog voltage using RC filter.

BLOCK H - VARACTOR VERIFICATION

Theory :

Varactor diode are semiconductor devices whose capacitance changes depending on the reverse bias voltage applied to them. They are commonly used in applications like voltage controlled oscillators, RF tuning circuits, and frequency modulators.

The capacitance of a varactor diode depends on the width of the depletion region at the PN junction. When a reverse bias voltage is applied, the depletion region becomes wider, which reduces the capacitance. On the other hand, when the reverse bias voltage is decreased, the depletion region narrows, causing the capacitance to increase.

This relationship between voltage and capacitance is not linear and is usually shown using a C-V characteristic curve in the diode specification sheets. Engineers use this curve to find the correct bias voltage needed to achieve a specific capacitance value.

In this project, the verification step focuses on making sure that the generated bias voltages match the intended capacitance values. Due to hardware limitations, the actual capacitance is not measured using an LCR meter. Instead, the relationship is confirmed theoretically by showing that the system can produce controlled voltages using PWM.

This stage is important because it supports the main goal of the system, which is to accurately control capacitance by adjusting the applied voltage.

Table :

PWM	Voltage	Capacitance
0	0V	High
128	2.5V	Medium
255	5V	Low

RESULTS

The implemented system successfully demonstrated how to remote a multi-channel capacitance control system works through an Arduino-based simulation. On the transmitter side, we managed to generate 48 independent digital values to represent various bias voltages. These were then converted into binary, effectively replicating the role of an Analog-to-Digital Converter (ADC).

The encoded bitstream was sent via serial communication, which acted as a streamlined model for digital modulation. The receiver performed perfectly, reconstructing the original data without any loss and proving that our approach to demodulation and bit recovery works under ideal conditions.

The highlights of the result came from the **PWM based voltage generation**. By adjusting the PWM duty cycle-testing levels like 0, 128, 255 - we were able to validate the resulting output voltages. The **RC low-pass filter** performed exactly as expected, smoothing the PWM signal into a stable, near-DC voltage and confirming that PWM is a highly effective alternative to a dedicated DAC.

While we physically built out one channel for this demonstration, the underlying architecture is fully modular. By integrating external PWM driver ICs, the system could easily expand to handle the full 48 channels. Ultimately, this project confirms that digital communication and analog voltage control can be continuously integrated to manage complex, multi-channel systems.

ADVANTAGES

One of the biggest strengths of this system is how easily it grows. While we started with a single channel prototype, the design is built for scalability-meaning it can be expanded to 48 or even more channels by using PWM expansion technique or dedicated driver chips. This flexibility makes it a perfect fit for high-demand setups like RF tuning networks or phased array antennas.

It's also incredibly cost-effective. Instead of spending a premium on specialized DAC (Digital-to-Analog Converter) hardware, we used a standard Arduino and simple RC filter components. Using PWM to generate voltage isn't just a workaround; it's a reliable, budget-friendly alternative for producing analog signals.

Beyond the hardware, the system is highly **adaptable**. Because the control logic lives in the software, you can modify, update, or reconstruct the system's behaviour without needing to pick a soldering iron. Whether you need to change a communication protocol or try a new modulation technique, a quick code update is usually all it takes.

Finally, this project is a great example of **interdisciplinary design**. It brings together digital electronics, communication theory, and analog circuits into one cohesive unit, making it an ideal platform for both rapid prototyping and hands-on learning.

LIMITATIONS

The implementation was successful; however, there remain several areas where the system can be further refined and enhanced. The most apparent challenge was the hardware bottleneck: because an Arduino UNO has a limited number of PWM pins, we could only physically build out one channel. The other 47 channels exists only in simulation for now, rather than as a full-scale hardware reality.

We also relied on a simplified communication model. Using serial communication allowed us to prove the concept, but it doesn't quite capture the messy reality of digital modulation. In a real-world environment, we'd have to contend with noise, signal interference, and attenuation factor that weren't present in our controlled setup.

The PWM - to - Analog approach is a great budget-saver, but it does come with a trade-off in precision. Since the output voltage depends entirely on the RC filter's quality, there's always a tiny bit of "ripple" or fluctuation. For high-stakes applications that need a perfectly steady voltage, this might need more refining.

Finally, our results are currently theoretical when it comes to the actual hardware performance. We've mapped out how voltage should control capacitance, but we haven't yet verified those numbers using an LCR mater or real-world varactor diode measurements. Bridging that gap between the math and the physically components would be the next logical setup.

CONCLUSION

This project successfully proves that we can manage remote, multi-channel capacitance control by combining digital communication with PWM techniques. By integrating different functional blocks-like ADC encoding, digital transmission, and PWM based voltage generation we've created a complete snapshot of how modern electronic systems are designed and integrated.

Even through this is a a simplified version, it clearly validates that a micro controller-based approach is a practical way to handle multiple channels. Specifically, using PWM with RC filtering stood out as a smart, low-cost solution for generating the analog voltages needed for the system.

One of the biggest takeaways here was the importance of system-level thinking. This project wasn't just about one component; it was about getting digital, analog, and communication domains to work together toward a single goal. It provides a solid foundation for future exploration into real-time communication, RF design, and even more complex control architectures.

Ultimately, the system does exactly what it set out to do. It offers a adaptable, programmable way to control capacitance that is as relevant in a classroom setting as it is for practical, real-world engineering.

FUTURE SCOPE

While this project successfully proves the core concept, there is plenty of room to take it from a simulation to a fully realized, professional-grade system.

One of the most immediate upgrades would be completing the hardware. By adding dedicated PWM driver ICs (like the PCA9685) or external DAC modules, we could move past the Arduino's pin limits and physically control all 48 channel at once. This would make the system ready for real-world RF environments where simultaneous, high-precision control is a must.

To make the communication more robust, the next step would be moving away from simple serial cables and implementing actual digital modulation-think FSK, PSK, or QPSK. Integrating these modern communication tech actually operates in the field.

We could also close the gap between theory and reality by testing with physical varactor diodes. Using an LCR meter or impedance analyzer to measure actual capacitance changes would give us the data needed to refine our models. Taking it a step further, we could even add a feedback loop that automatically adjusts the voltage to hit a specific capacitance target, ensuring the system stays accurate even if conditions change.

Looking ahead, this architecture has huge potential for high-end tech like adaptive antenna arrays, programmable RF filters, and phased-array radar. By upgrading to more powerful micro controllers and smarter protocols, this project could evolve from a foundational prototype into a fully automated, intelligent control platform.

REFERNCES

Bringing this project to a life was a collaborative effort between theory and practice, drawing from a mix of technical manuals, classic textbooks, and online engineering communities.

The Arduino official documentation was our go-to manual for the heavy lifting in software. It provided the essential blueprints for PWM generation, serial communication, and the overall programming logic that runs the micro controller.

To get the theory right, we leaned on the “bibles” of the field. Concepts like modulation, demodulation, and channel modeling were grounded in the principles found in **Simon Haykin`s Digital Communications** and **B.P. Lathis`s Communication Systems**. These texts were instrumental in helping us translate abstract encoding and transmission ideas into a working digital model.

When it came to the hardware specifics-especially understanding how varactor diodes behave we went straight to the source. Manufacturer data sheets for the **BB135** and **SMV1231** were crucial for mapping out exactly how much capacitance we could expect at different voltage levels.

Finally, a variety of academic articles and online tutorials filled in the gaps for the more specialized hardware tweaks, like optimizing the **RC filter design** and perfecting the PWM based DAC. Together, these resources provided the roadmap we needed to move from a basic idea to a successful, functioning system.

APPENDIX

```
Int values[48] = {
  10,20,30,40,50,60,70,80,
  90,100,110,120,130,140,150,160,
  170,180,190,200,210,220,230,240,
  50,60,70,80,90,100,110,120,
  130,140,150,160,170,180,190,200,
  210,220,230,240,250,255,100,50
};
Int pwmPin = 9;
Void setup() {
   Serial.begin(9600);
  pinMode(pwmPin, OUTPUT);
}
Void loop() {
// TRANSMISSION
  Serial.println("START_FRAME");
  for(int i = 0; i < 48; i++) {
    Serial.print(values[i], BIN);
    Serial.print(" ");
   }
  Serial.println();
  Serial.println("END_FRAME");
  // RECEIVER (SIMULATED)
  Serial.println("RECEIVER START");
  for(int i = 0; i < 48; i++) {
    int receicedValue = values[i];
    Serial.print("channel");
    Serial.print(i+1);
    Serial.print(": ");
    Serial.println(receivedValue);
  }
```

```
Serial.println("RECEIVER END");
  // PWM OUTPUT
  int pwmValue = 128;
  analogWrite(pwmPin, pwmValue);
  Serial.print("PWM Value: ");
  Serial.println(pwmValue);
  Delay(4000);
}
```

THE END